LAPORAN PRAKTIKUM
INTERNET OF THINGS (IOT) 2

Integrasi Backend Server Lokal (Docker & Mosquitto) dan Rule Engine (Node-RED)

Dosen pengampu:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.,
Agus Ambarwari, S.Pd., M.Kom.,
Andi Saptono, S.Kom

Disusun Oleh:

IMELDA NURAINI (25782041)

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2025/2026

7.6 Latihan

Latihan 1 (Analisis Jaringan Internal Docker & Service Discovery):

Pada Praktikum 3, di dalam konfigurasi node mqtt in pada Node-RED, kita mengisi alamat server broker dengan nama teks **mqtt**, bukan alamat IP lokal laptop (192.168.1.15) maupun localhost.

i. Mengapa penulisan localhost pada Node-RED di dalam kontainer Docker akan menyebabkan kegagalan koneksi (*Connection Refused*)?

Jawab:

Di dalam ekosistem Docker, setiap kontainer berjalan terisolasi dengan menggunakan network namespace (ruang nama jaringan) yang mandiri. Apabila Anda menuliskan localhost atau 127.0.0.1 di dalam konfigurasi kontainer Node-RED, sistem operasi akan menerjemahkannya ke antarmuka loopback milik kontainer Node-RED itu sendiri, bukan ke laptop induk (Host) ataupun kontainer lain. Akibatnya, karena layanan broker MQTT (Eclipse Mosquitto) berjalan di kontainer terpisah (mqtt-broker), tidak ada layanan broker yang aktif mendengarkan pada port 1883 di dalam loopback kontainer Node-RED tersebut, sehingga koneksi ditolak dan memunculkan galat Connection Refused.

ii. Jelaskan bagaimana mekanisme *Docker Embedded DNS Server* dan *Bridge Network* bekerja sehingga kontainer node-red dapat menemukan kontainer mqtt-broker cukup dengan memanggil nama layanannya!

Jawab:

Ketika menggunakan docker-compose.yml, Docker secara otomatis menyediakan sebuah jaringan kustom bertipe Bridge Network yang menghubungkan seluruh kontainer dalam satu orkestrasi. Di dalam jaringan bridge ini, Docker mengaktifkan layanan Embedded DNS Server internal yang bekerja secara transparan. Saat kontainer node-red mencoba menghubungi alamat menggunakan nama teks mqtt (yang merepresentasikan nama layanan mqtt / container_name: mqtt-broker), permintaan pencarian nama tersebut dicegat oleh Docker Embedded DNS Server untuk dipetakan secara langsung ke alamat IP internal kontainer mqtt-broker yang valid di dalam jaringan tersebut. Dengan mekanisme service discovery ini, kontainer Node-RED dapat berkomunikasi secara mulus dengan kontainer Mosquitto tanpa perlu mengetahui alamat IP internal kontainer yang sifatnya dinamis.

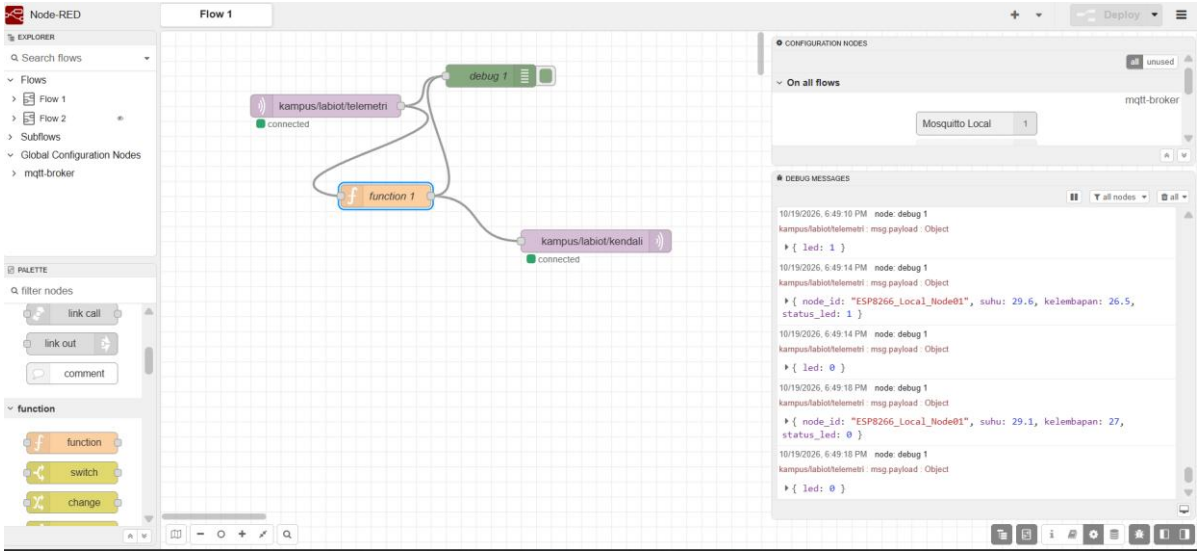
Latihan 2 (Penanganan Masalah Konektivitas Jaringan & Subnet Firewall):
Seorang mahasiswa mencoba menghubungkan NodeMCU ESP8266 ke broker lokal di laptopnya. ESP8266 berhasil terhubung ke Wi-Fi dan mendapatkan alamat IP 192.168.43.102, sedangkan laptop memiliki alamat IP 192.168.1.15.

i. Mengapa NodeMCU tidak akan pernah berhasil memublikasikan data ke broker Mosquitto pada kondisi tersebut?
Jawab:
NodeMCU ESP8266 dan laptop berada pada dua segmen jaringan atau subnet yang berbeda, yaitu NodeMCU berada pada subnet 192.168.43.x (biasanya berasal dari tethering perangkat seluler), sementara laptop berada pada subnet 192.168.1.x (dari router Wi-Fi lokal atau jaringan kabel yang berbeda). Karena berada di subnet yang berlainan dan tidak berada dalam satu domain siaran (broadcast domain), kedua perangkat tidak dapat saling merutekan paket data secara langsung tanpa adanya perangkat router pihak ketiga atau aturan routing khusus. Akibatnya, paket MQTT yang dikirimkan oleh NodeMCU tidak akan pernah sampai ke alamat IP laptop (192.168.1.15) tempat broker Mosquitto berjalan, sehingga komunikasi gagal total.

ii. Langkah teknis apa yang harus dilakukan mahasiswa tersebut agar kedua perangkat berada dalam domain *broadcast / subnet* yang sama?
Jawab:
Mahasiswa harus memastikan bahwa NodeMCU ESP8266 dan laptop terhubung ke titik akses (Access Point) atau sumber jaringan Wi-Fi yang sama persis. Langkah praktisnya adalah mengubah konfigurasi Wi-Fi pada laptop agar terhubung ke jaringan hotspot seluler yang sama dengan NodeMCU (sehingga laptop mendapatkan IP dengan segmen 192.168.43.x), atau sebaliknya, menyambungkan NodeMCU ke jaringan Wi-Fi utama tempat laptop terhubung. Selain itu, pastikan fitur AP Isolation atau Client Isolation pada pengaturan router atau hotspot dalam kondisi nonaktif (off) agar kedua perangkat di dalam jaringan lokal tersebut diizinkan untuk saling berkomunikasi secara langsung.

Latihan 3 (Implementasi Filter Histeresis pada Rule Engine):
Pada sistem kendali dua posisi (*Bang-Bang*), jika suhu berfluktuasi cepat di sekitar ambang batas 31.0°C(misal: 31.05°C → 30.95°C → 31.02°C), lampu LED indikator akan berkedip-kedip cepat (*flickering / hunting*) yang mengganggu stabilitas visual sistem.
Rancang modifikasi alur Node-RED menggunakan **function node** berbahasa JavaScript yang menerapkan logika **Histeresis**:

- LED baru menyala jika suhu > 31.5°C(msg.payload = {"led": 1}).
- LED baru padam jika suhu mendingin hingga < 30.0°C(msg.payload = {"led": 0}).
  Tuliskan kode JavaScript yang Anda masukkan ke dalam *Function Node* tersebut!



Pada latihan implementasi filter histeresis ini, alur logika pada *Node-RED* dirancang menggunakan *function node* berbasis JavaScript untuk mencegah terjadinya kedipan cepat atau *hunting* pada aktuator LED akibat fluktuasi suhu yang berada tepat di sekitar ambang batas 31.0°C. Berdasarkan hasil pengujian pada sistem yang berjalan, fungsi ini menggunakan area *deadband* dengan menetapkan batas atas di atas 31.5°C untuk menyalakan LED dan batas bawah di bawah 30.0°C untuk memadamkannya.
Hal ini terbukti sangat efektif pada panel *Debug Messages*, di mana sistem mampu menahan status aktuator dengan stabil ketika suhu berada di antara rentang tersebut. Ketika suhu ruangan naik melewati 31.5°C, sistem secara konsisten mengeluarkan instruksi { led: 1 }, dan sebaliknya otomatis mengubah *payload* menjadi { led: 0 } hanya ketika suhu benar-benar mendingin di bawah 30.0°C, sehingga kestabilan visual sistem kendali dapat terjaga dengan baik.

Latihan 4 (Analisis Mekanisme Volume Mount & Persistensi Data):
Pada berkas docker-compose.yml, terdapat pemetaan direktori - ./nodered:/data dan - ./mosquitto/data:/mosquitto/data.

i. Apa yang akan terjadi pada alur kanvas visual Node-RED yang telah Anda buat jika kontainer dimatikan dengan perintah docker compose down, kemudian dinyalakan kembali, seandainya baris volume tersebut dihapus dari docker-compose.yml?

Jawab:

Jika baris pemetaan direktori (*volume*) dihapus dari berkas docker-compose.yml dan kontainer dimatikan menggunakan perintah docker compose down, seluruh alur kanvas visual (*flow*), node, serta data konfigurasi yang telah Anda buat di dalam Node-RED akan **hilang total dan kembali ke kondisi awal yang kosong**. Hal ini terjadi karena siklus hidup data kontainer secara *default* bersifat sementara (*ephemeral*); ketika kontainer dihapus, seluruh penyimpanan internal di dalamnya turut terhapus. Dengan adanya konfigurasi *volume mount*, direktori penyimpanan kontainer dipetakan ke direktori komputer lokal (*host*), sehingga data alur kerja tetap aman dan persisten meskipun kontainer dimatikan atau dihapus.

ii. Jelaskan perbedaan mendasar antara *Bind Mount* dan *Named Volume* pada Docker!

Jawab:

ind Mount: Merupakan metode pemetaan di mana direktori atau berkas apa pun yang berada di komputer induk (*host machine*) dapat langsung dipasang (*mount*) ke dalam kontainer. Lokasi penyimpanan ditentukan secara spesifik menggunakan jalur absolut atau relatif pada sistem *host* (contohnya `./nodered:/data`), sehingga pengguna dapat dengan mudah membaca dan menyunting berkas proyek secara langsung dari luar kontainer menggunakan editor teks biasa.

Named Volume: Merupakan ruang penyimpanan khusus yang dikelola sepenuhnya oleh Docker dan diisolasi dari direktori sistem berkas *host* utama. Docker membuat area penyimpanan mandiri di dalam sistem internalnya (biasanya di `/var/lib/docker/volumes/`) tanpa mengharuskan pengguna menentukan jalur folder di komputer *host*. Metode ini jauh lebih aman dan direkomendasikan untuk manajemen basis data produksi karena lokasinya tersembunyi dan dikelola secara otomatis oleh Docker Engine.